PROCESS SCHEDULING:

The **process scheduler** selects an available process for program execution on the CPU.

- For a single-processor system, there will never be more than one running process. If there are more processes, the rest will have to wait until the CPU is free and can be rescheduled.
- The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization.
- The objective of time sharing is to switch the CPU among processes so frequently that users can interact with each program while it is running.

Scheduling Queues:

- The Scheduling Queues are of three types
  - Job Queue
  - Ready Queue
  - Device Queue
- As processes enter the system, they are put into a **job queue**, which consists of all processes in the system.
- The processes that are residing in main memory and are ready and waiting to execute are kept on a list called the **ready queue**
- A ready-queue header contains pointers to the first and final PCBs in the list.
- Each process that requires I/O Operation may have to wait for the device. The list of processes waiting for a particular I/O device is called a **device queue**.

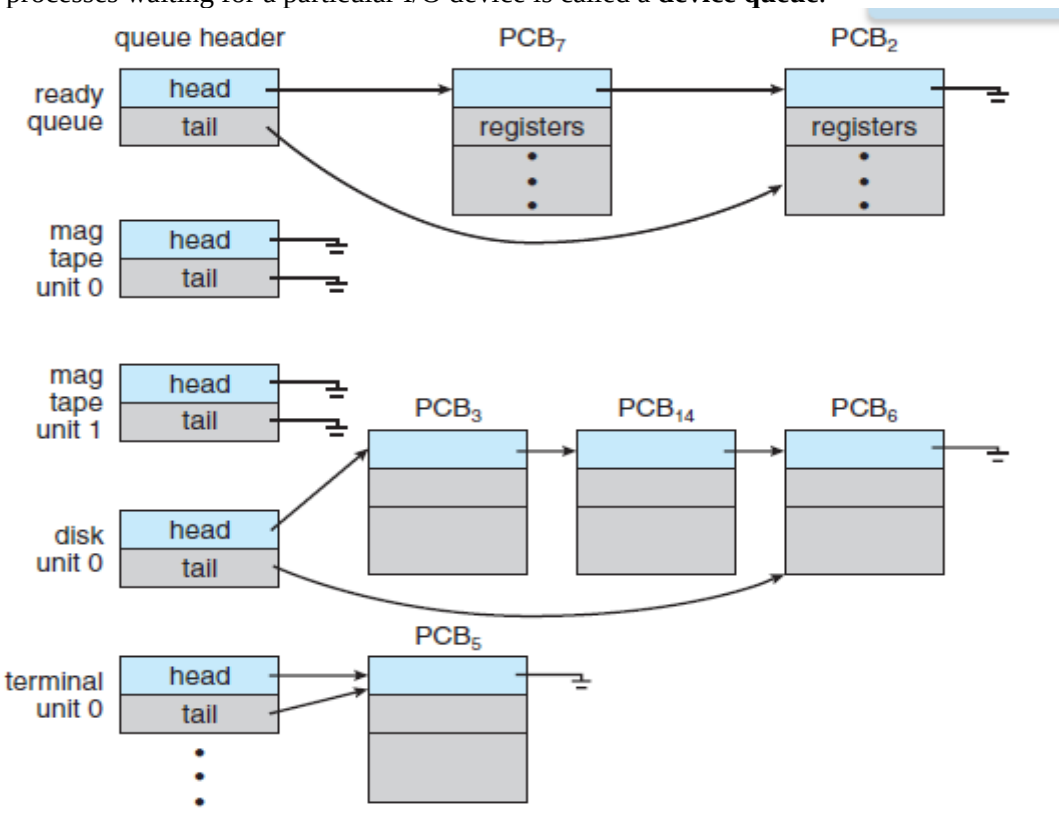


Figure 3.5 The ready queue and various I/O device queues.

- A common representation of process scheduling is a **queuing diagram**
- Two types of queues are present: **Ready queue and a set of device queues.**
- The circles represent the resources that serve the queues, and the arrows indicate the flow of processes in the system.

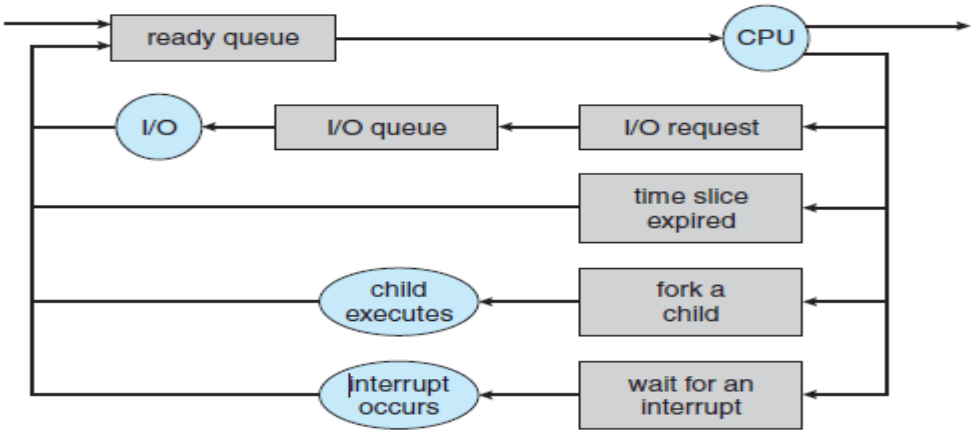


Figure 3.6 Queueing-diagram representation of process scheduling.

- A new process is initially put in the ready queue. It waits there until it is selected for execution, or **dispatched.**
- Once the process is allocated the CPU and is executing, one of several events could occur:

• The process could issue an I/O request and then be placed in an I/O queue.
• The process could create a new child process and wait for the child's termination.
• The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue.

Schedulers:

- The operating system must select, for scheduling purposes, processes from the queues in some approach. The selection process is carried out by the appropriate **scheduler**.
- It makes use of two types of schedulers
  - Long term scheduler or job scheduler
  - Short term scheduler or CPU scheduler.
  - Medium term scheduler
- The **long-term scheduler**, or **job scheduler**, selects processes from the job queue and loads them into memory for execution.
- The **short-term scheduler**, or **CPU scheduler**, selects from among the processes that are ready to execute and allocates the CPU to one of them.
- The Long term scheduler must have a careful selection of both I/O Bound and CPU Bound process.
- An **I/O-bound process** is one that spends more of its time doing I/O than it spends doing computations.
- A **CPU-bound process**, in contrast, generates I/O requests infrequently, using more of its time doing computations.
- If all processes are I/O bound, the ready queue will almost always be empty, If all processes are CPU bound, the I/O waiting queue will almost always be empty, devices will go unused.
- The medium-term scheduler is used to remove a process from memory to reduce the degree of multiprogramming. Later, the process can be reintroduced into memory, and its execution can be continued where it left off. This scheme is called **swapping**.
- The process is swapped out, and is later swapped in, by the medium-term scheduler.

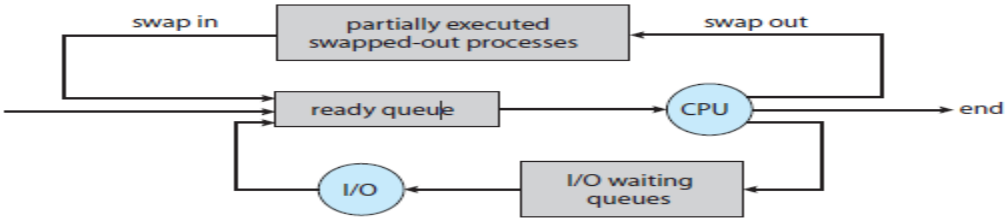


Figure 3.7 Addition of medium-term scheduling to the queueing diagram.

Context Switch:

- The process of switching the CPU from one process to another process requires performing a state save of the current process and a state restore of a different process. This task is known as a **context switch.**
- When an interrupt occurs, the system needs to save the current **context** of the process running on the CPU so that it can restore that context when its processing is done.
- The context is represented in the PCB of the process. It includes the value of the CPU registers, the process state and memory management information.

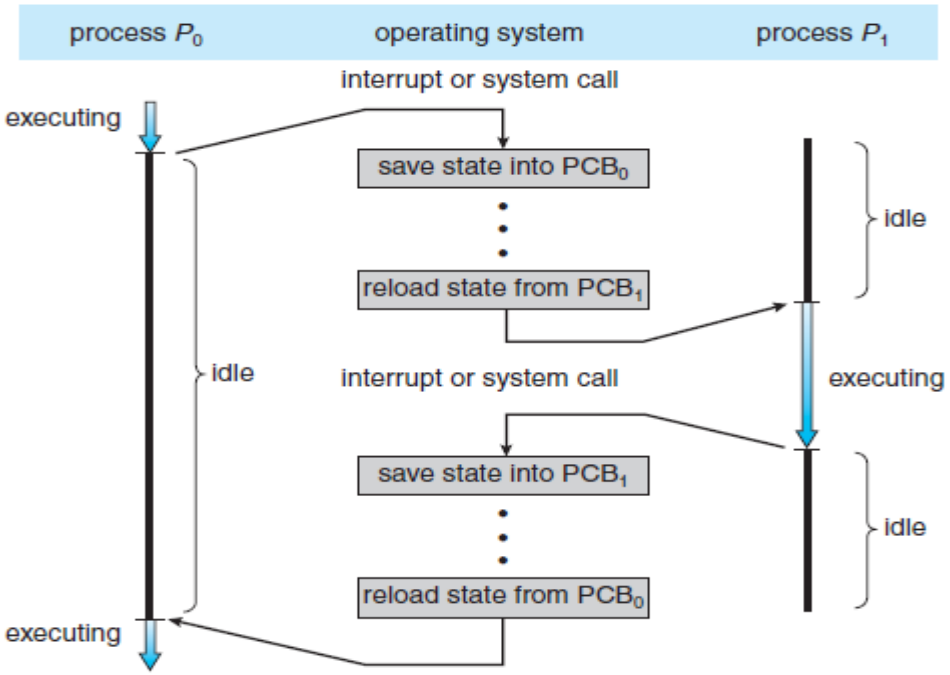


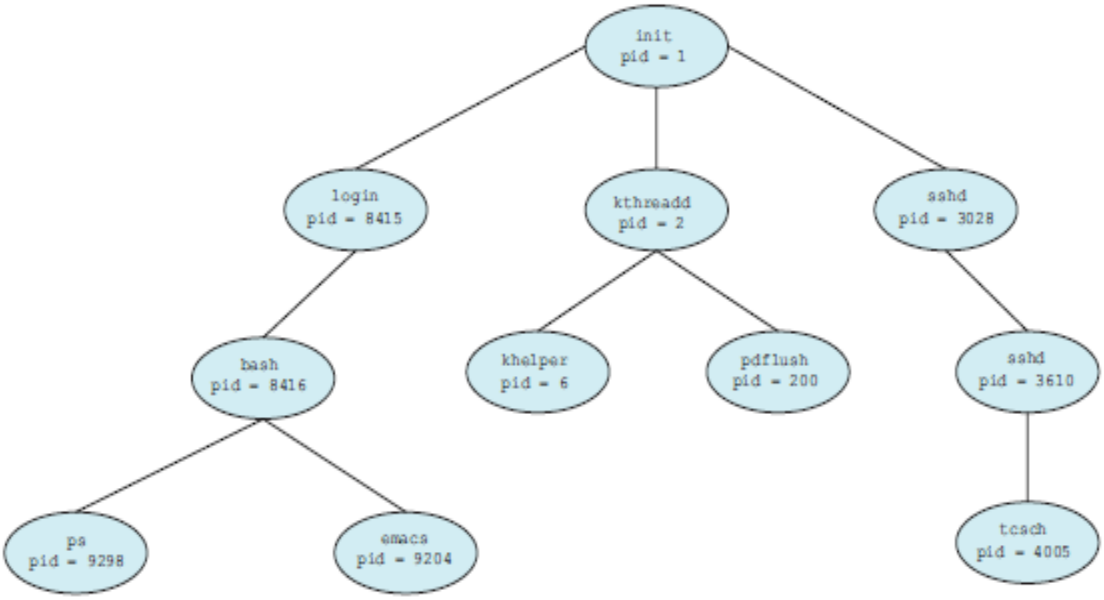
Figure 3.4 Diagram showing CPU switch from process to process.

OPERATIONS ON PROCESSES:

- The operating system must provide a mechanism for process creation and termination. The process can be created and deleted dynamically by the operating system.
- The Operations on the process includes
  - Process creation
  - Process Termination

Process Creation:

- During Execution a process may create several new processes.
- The creating process is called as the **parent process** and the newly created process is called as the **child process**.
- The operating systems identify the processes according to their unique process identifier.
- **Example: Process tree for Linux operating system**.



- The init process serves as the root parent process for all the user process.
- Once the system has booted, the init process can also create various user processes, such as a web or print server, an ssh server.
- The kthreadd process is responsible for creating additional processes that perform tasks on behalf of the kernel
- The sshd process is responsible for managing clients that connect to the system by using ssh(Secure shell)
- The login process is responsible for managing clients that directly log onto the system
- The command ps –el will list complete information for all processes currently active in the system.
- **When a process creates a new process, two possibilities for execution exist**:
  1. The parent continues to execute concurrently with its children.
  2. The parent waits until some or all of its children have terminated
- **There are also two address-space possibilities for the new process:**
  1. The child process is a duplicate of the parent process (it has the same program as the parent).
  2. The child process has a new program loaded into it.
- A new process is created by the fork() system call. The new process consists of a copy of the address space of the original process. This mechanism allows the parent process to communicate easily with its child process.
- **The return code for the fork() is zero for the new (child) process, whereas the (nonzero) process identifier of the child is returned to the parent.**
- After a fork() system call, one of the two processes typically uses the exec() system call to replace the process's memory space with a new program.

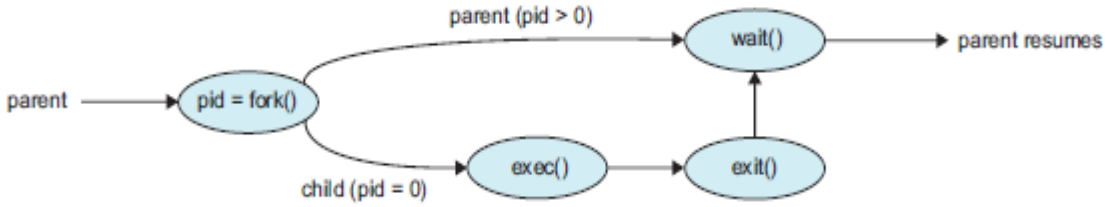


Figure 3.10 Process creation using the `fork()` system call.

Creating a separate process using the UNIX fork() system call:

- The parent and child are concurrent processes running the same code instructions. Because the child is a copy of the parent, each process has its own copy of any data.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int main()
{
Pid_t pid;
/* fork a child process */
pid = fork();
if (pid < 0) { /* error occurred */
fprintf(stderr, "Fork Failed");
return 1;
}
else if (pid == 0) { /* child process */
execlp("/bin/ls","ls",NULL);
}
else { /* parent process */
/* parent will wait for the child to complete */
wait(NULL);
printf("Child Complete");
}
return 0;
}
```

Process Termination:

- A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the exit() system call.
- At that point, the process may return a status value (typically an integer) to its parent process.
- All the resources of the process—including physical and virtual memory, open files, and I/O buffers—are deallocated by the operating system
- **A parent may terminate the execution of one of its children for a variety of reasons, such as**
  - The child has exceeded its usage of some of the resources that it has been allocated.
  - The task assigned to the child is no longer required.
  - The parent is exiting, and the operating system does not allow a child to continue if its parent terminates.
- Some systems do not allow a child to exist if its parent has terminated. In such systems, if a process terminates (either normally or abnormally), then all its children must also be terminated. This phenomenon is referred to as **cascading termination.**
- A parent process may wait for the termination of a child process by using the wait() system call
- This system call also returns the process identifier of the terminated child so that the parent can tell which of its children has terminated:

```
pid t pid;
int status;
pid = wait(&status);
```

- A process that has terminated, but whose parent has not yet called wait(), is known as a **zombie** process.

INTERPROCESS COMMUNICATION:

- A process can be either an independent process or a cooperating process.
- A process is independent if it cannot affect or be affected by the other processes executing in the system. Any process that does not share data with any other process is independent.
- A process is cooperating if it can affect or be affected by the other processes executing in the system. Clearly, any process that shares data with other processes is a cooperating process.
- Advantages of cooperating process
  i) Information sharing
  ii) Computation speedup
  iii) Modularity
  iv) Convenience.

- **DEFINITION: An Interprocess communication is a mechanism that allows the cooperating process to exchange data and communication among each other.**
- There are two fundamental models of Interprocess communication
  - **Shared Memory model**
  - **Message passing model**
- In shared memory model a region of memory is shared by the cooperating process. Processes can then exchange information by reading and writing data to the shared region.
- In the message-passing model, communication takes place by means of messages exchanged between the cooperating processes.
- Message passing is also easier to implement in a distributed system than shared memory.
- The shared memory is faster than that of message passing.

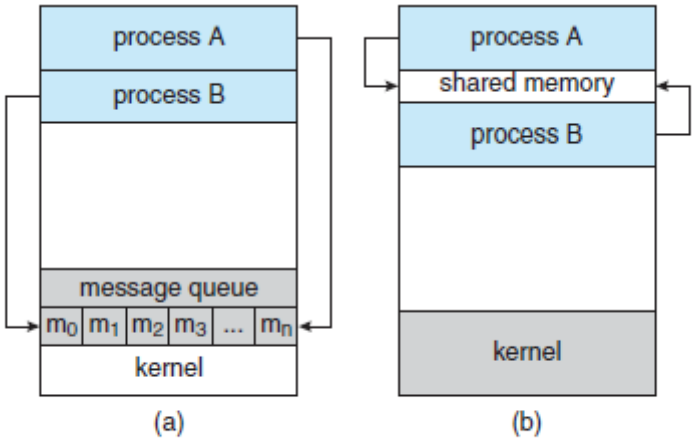


Figure 3.12 Communications models. (a) Message passing. (b) Shared memory.

Shared-Memory Systems:

- Interprocess communication using shared memory requires communicating processes to establish a region of shared memory.
- Shared-memory region resides in the address space of the process creating the shared-memory segment.
- Other processes that wish to communicate using this shared-memory segment must attach it to their address space.
- They can then exchange information by reading and writing data in the shared areas.
- **EXAMPLE:PRODUCER – CONSUMER PROCESS:**
- A **producer** process produces information that is consumed by a **consumer** process.
- One solution to the producer–consumer problem uses shared memory
- To allow producer and consumer processes to run concurrently, we must have available a buffer of items that can be filled by the producer and emptied by the consumer.
- This buffer will reside in a region of memory that is shared by the producer and consumer processes. A producer can produce one item while the consumer is consuming another item.

- Two types of buffers can be used.
  - Bounded Buffer.
  - Unbounded Buffer.
- The **unbounded buffer** places no practical limit on the size of the buffer. The consumer may have to wait for new items, but the producer can always produce new items.
- The **bounded buffer** assumes a fixed buffer size. In this case, the consumer must wait if the buffer is empty, and the producer must wait if the buffer is full.
- The following variables reside in a region of memory shared by the producer and consumer processes:

```
#define BUFFER SIZE 10
typedef struct {
. . .
}item;
item buffer[BUFFER SIZE];
int in = 0;
int out = 0;
```

- The variable in points to the next free position in the buffer; out points to the first full position in the buffer. The buffer is empty when in == out; the buffer is full when ((in + 1) % BUFFER SIZE) == out.
- **CODE FOR PRODUCER PROCESS:**

```
item next produced;
while (true) {
/* produce an item in next produced */
while (((in + 1) % BUFFER SIZE) == out)
; /* do nothing */
buffer[in] = next produced;
in = (in + 1) % BUFFER SIZE;
}
```

- The producer process has local variable next produced in which the new item to be produced is stored.
- The consumer process has a local variable next consumed in which the item to be consumed is stored.
- This scheme allows at most BUFFER SIZE − 1 items in the buffer at the same time.
- **CODE FOR CONSUMER PROCESS**

```
item next consumed;
while (true) {
while (in == out)
; /* do nothing */
next consumed = buffer[out];
out = (out + 1) % BUFFER SIZE;
/* consume the item in next consumed */
}
```

Message-Passing Systems:

- Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space
- A message-passing facility provides at least two operations:
  - Send(message)
  - Receive(message)
- If processes P and want to communicate, they must send messages to and receive messages from each other: a **communication link** must exist between them.
- There are several methods for logically implementing a link and the send()/receive() operations:
  - Direct or indirect communication
  - Synchronous or asynchronous communication
  - Automatic or explicit buffering

Direct or indirect communication

Direct Communication:

- Each process that wants to communicate must explicitly name the recipient or sender of the communication.
- Direct communication can be done in two ways symmetric addressing and asymmetric addressing.
- In Symmetric addressing both the sender process and the receiver process must name the other to communicate.
  - send(P, message)—Send a message to process P.
  - receive(Q, message)—Receive a message from process Q.
- In Asymmetric addressing only the sender names the recipient; the recipient is not required to name the sender.
  - send(P, message)—Send a message to process P.
  - receive(id, message)—Receive a message from any process
- A communication link in this scheme has the following properties:
  • A link is established automatically between every pair of processes that want to communicate. The processes need to know only each other's identity to communicate.
  • A link is associated with exactly two processes.
  • Between each pair of processes, there exists exactly one link.

Indirect communication:

- With **indirect communication**, the messages are sent to and received from **mailboxes**, or **ports**.
- A mailbox can be viewed abstractly as an object into which messages can be placed by processes and from which messages can be removed. Each mailbox has a unique identification
- The send() and receive() primitives are defined as follows:
  - send(A, message)—Send a message to mailbox A.
  - receive(A, message)—Receive a message from mailbox A.
- In this scheme, a communication link has the following properties:
  • A link is established between a pair of processes only if both members of the pair have a shared mailbox.
  • A link may be associated with more than two processes.
  • Between each pair of communicating processes, a number of different links may exist, with each link corresponding to one mailbox.
- A mailbox may be owned either by a process or by the operating system.

Synchronous or asynchronous communication

- Communication between processes takes place through calls to send() and receive() primitives.
- Message passing may be either **blocking** or **nonblocking**— also known as **synchronous** and **asynchronous**
- **Blocking send**. The sending process is blocked until the message is received by the receiving process or by the mailbox
- **Nonblocking send**. The sending process sends the message and resumes Operation
- **Blocking receive**. The receiver blocks until a message is available.
- **Nonblocking receive**. The receiver retrieves either a valid message or a null.
- When both send() and receive() are blocking, we have a **rendezvous** between the sender and the receiver.

Automatic or explicit buffering:

- Messages exchanged by communicating processes reside in a temporary queue. Basically, such queues can be implemented in three ways:
  - Zero capacity.
  - Bounded capacity

    - unbounded capacity

- **Zero capacity**. The queue has a maximum length of zero In this case, the sender must block until the recipient receives the message.
- **Bounded capacity**. The queue has finite length n; thus, at most n messages can reside in it.
- **Unbounded capacity**. The queue's length is potentially infinite; thus, any number of messages can wait in it. The sender never blocks.